Structure-Preserving Graph Representation Learning

Prof. O-Joun Lee

Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

네트워크 과학 연구실
NETWORK SCIENCE LAB

가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

Contents

- Overview of Structure-Preserving / Role-based GRL
- Feature-based models:
  - RolX
- Random-walk structural models:
  - Struc2vec
  - Role2vec
- Spectral structural models:
  - GraphWave

Recap: Representation Learning in Graphs

- Goal: Learn representation (features) for a set of graph elements (nodes, edges, etc.)

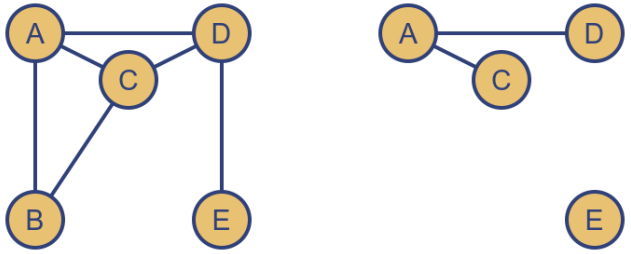
  Given $G = (V, E)$

  Learn a function $f: V \rightarrow \mathbb{R}^{\mathbb{d}}$

- Key intuition: Map the graph elements (e.g., nodes) to the d-dimension space
- Use the features for any downstream prediction task
- *Examples: deepWalk, node2vec, GCN, etc.*

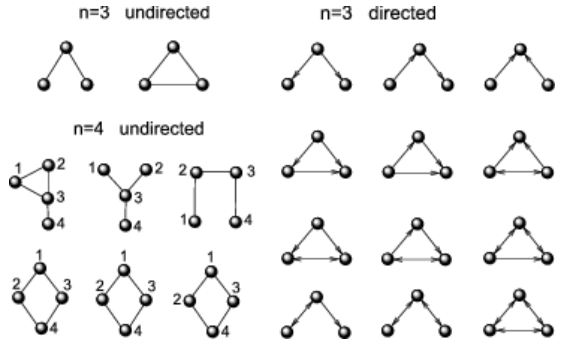
Graph Structure

- The graph structure can be sampled through connections between nodes in graphs or substructures

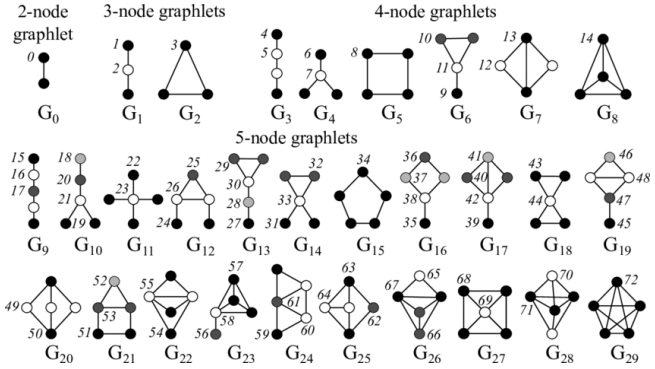


Graph G　　Subgraph of G

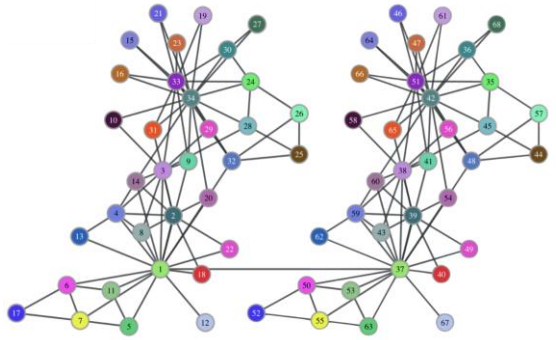
Subgraphs



Motifs



Graphlets

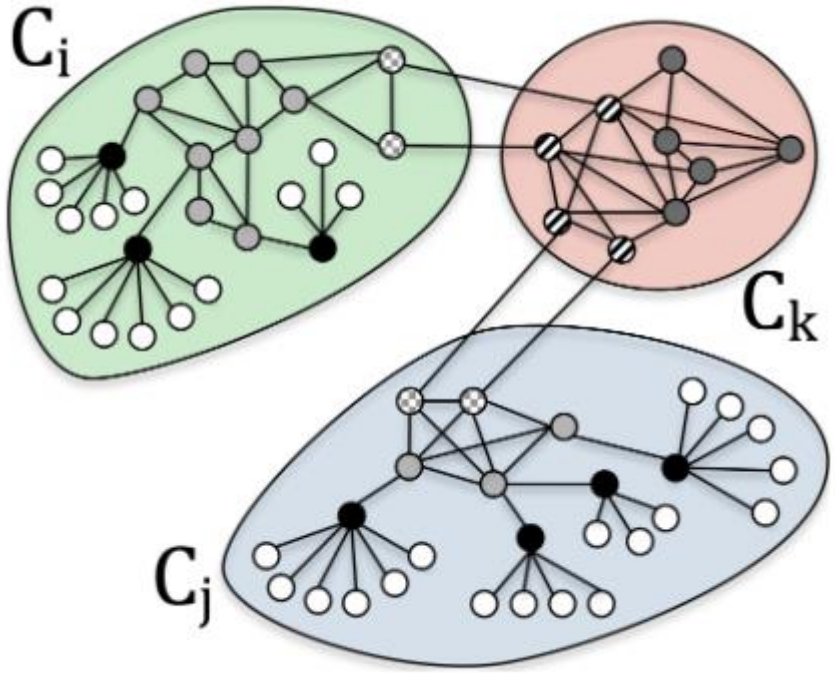


Roles

Reference: Hoang, Van Thuy, et al. "Graph representation learning and its applications: a survey." *Sensors* 23.8 (2023): 4168.

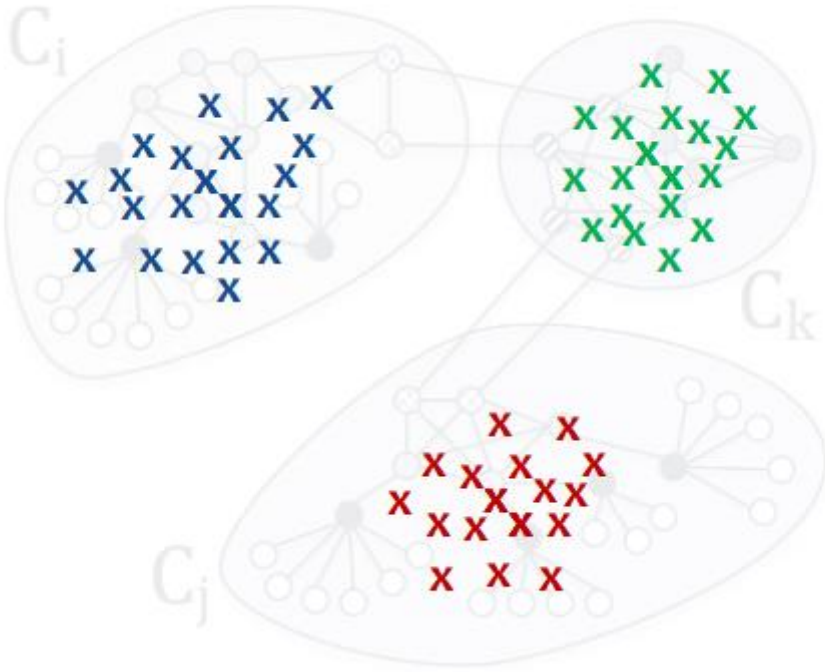
Two complimentary Notions in Graphs

- Proximity vs Structural Similarity



Two complimentary Notions in Graphs

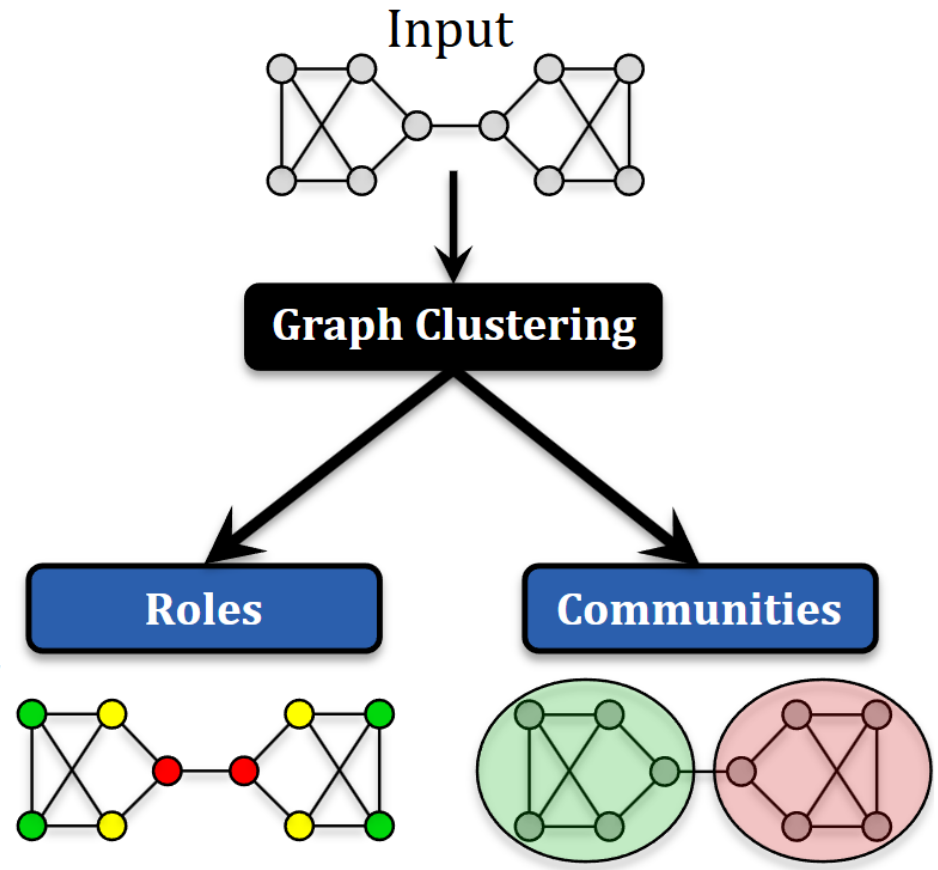
➢ Most existing work focuses on Modeling Graph Proximity



→ No guarantee that nearby vertices are structurally similar.

e.g., Deepwalk, GraRep, node2vec, Line, etc.

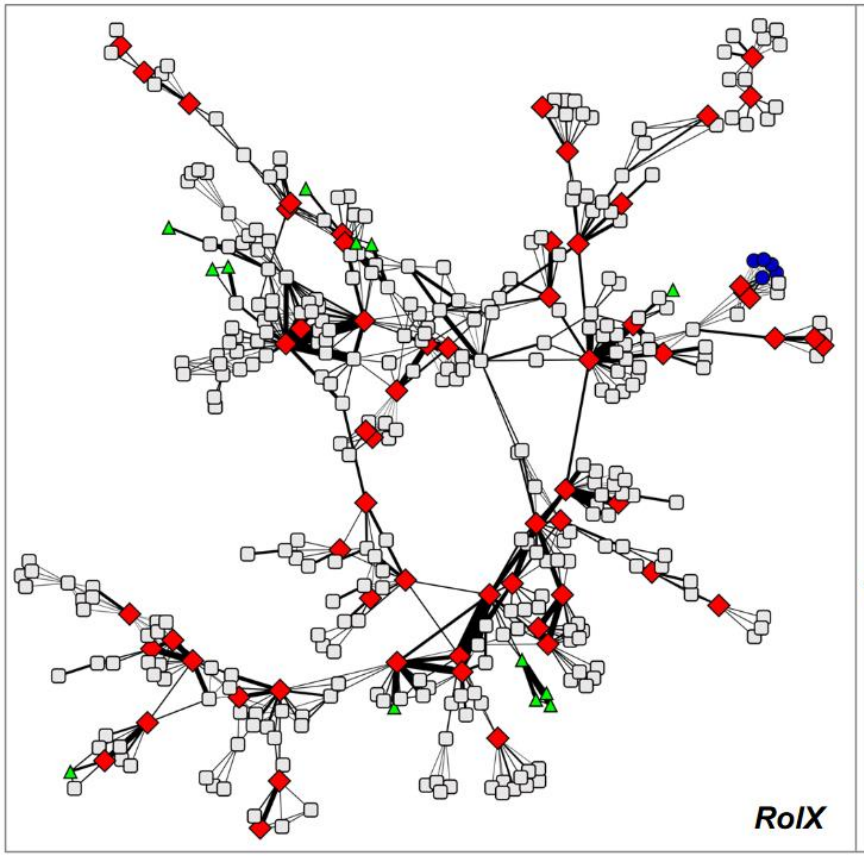
- **Roles** are sets of nodes that are more structurally similar to nodes inside the set than outside
  → Based on structural similarity
- **Communities** are sets of nodes with more connections inside the set than outside
  → Based on proximity, density
- Consider the social network of a CS Department:
  - Roles: Professors, Staff, Students
  - Communities: AI Lab, Info Lab, Theory Lab



References:
- Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, cs224w.stanford.edu
- Rossi, Ryan A., et al. "On proximity and structural role-based embeddings in networks: Misconceptions, techniques, and applications." ACM Transactions on Knowledge Discovery from Data (TKDD) 14.5 (2020): 1-37.

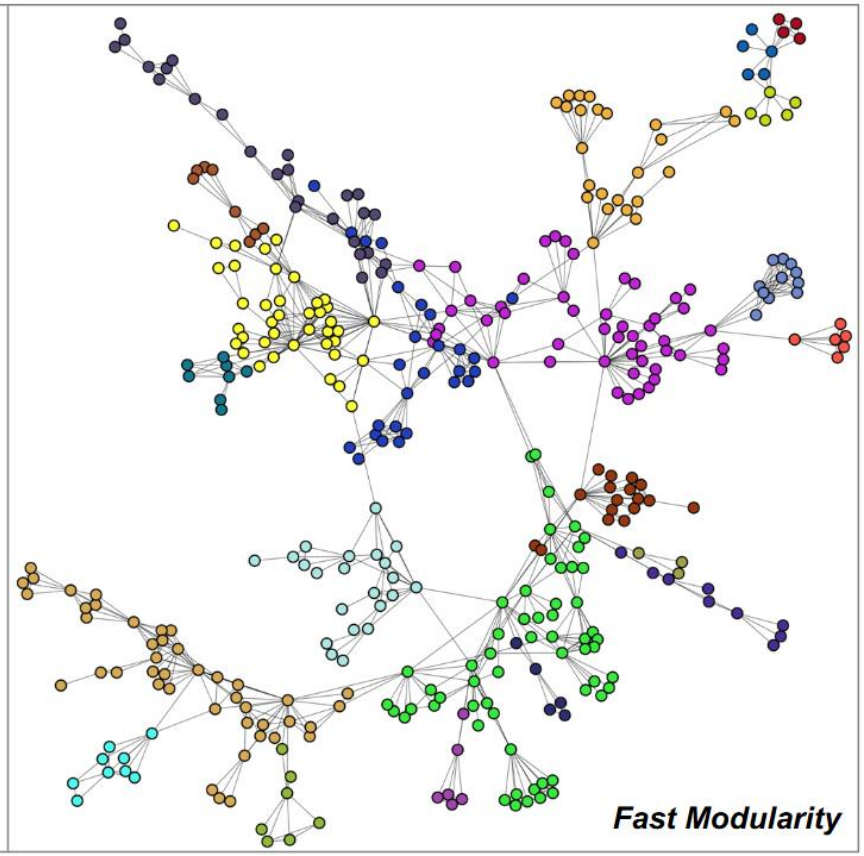
Communities and Roles

Roles



Henderson, et al., KDD 2012

Communities

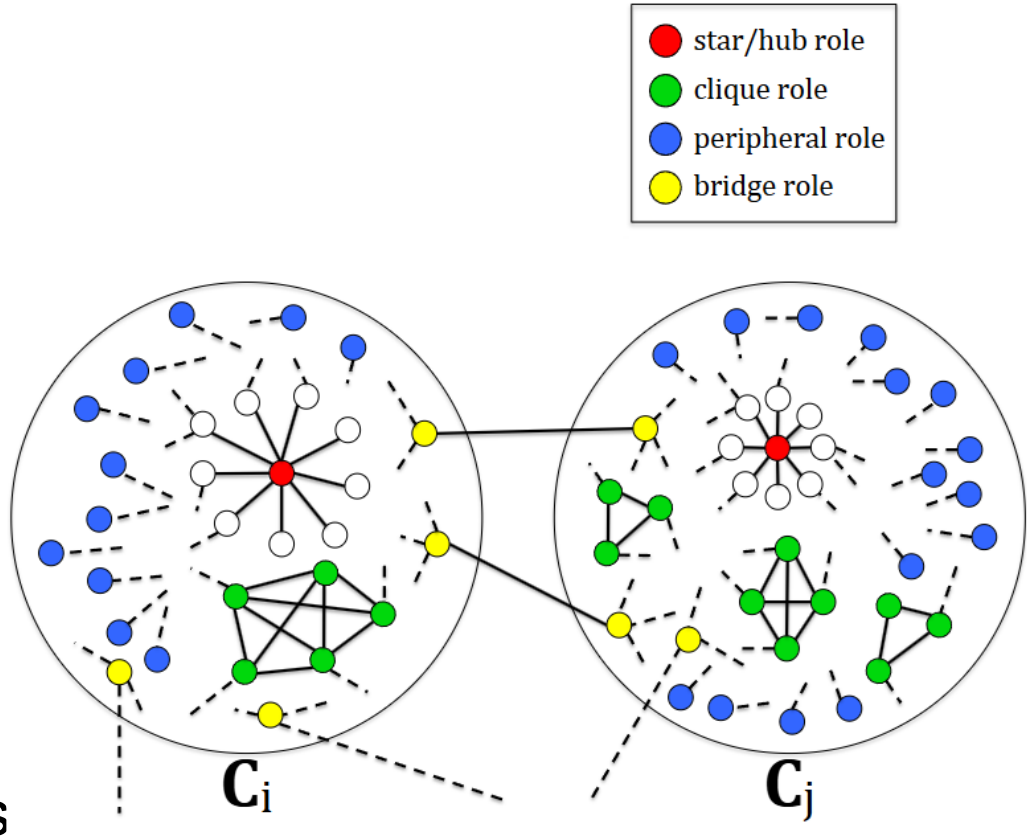


Clauset, et al., Phys. Rev. E 2004

Reference: J Rossi, Ryan A., et al. "On proximity and structural role-based embeddings in networks: Misconceptions, techniques, and applications." ACM Transactions on Knowledge Discovery from Data (TKDD) 14.5 (2020): 1-37.

Roles and Communities are ***Complementary***

- Roles based on structural similarity
  - Communities based on proximity, density
- Roles globally distributed
  - Communities are local
- Roles generalize
  - Communities do not generalize acros graphs (for graph transfer learning tasks)



Problems

Successes in Classification

Recent approaches like *DeepWalk* and *node2vec* excel in classification tasks.

They rely heavily on homophily - the tendency for nodes that are close in the network to share similar features (e.g., blogs with the same political inclination).

Failure in Structural Equivalence

If classification depends on structural identity rather than homophily, these approaches underperform.
Because they map proximity to closeness in the latent space, structurally identical nodes that are "far" in the network are erroneously separated in the latent representation.

References:

- Ribeiro, Leonardo FR, Pedro HP Saverese, and Daniel R. Figueiredo. "struc2vec: Learning node representations from structural identity." Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining. 2017.
- Ahmed, Nesreen K., et al. "Role-based graph embeddings." IEEE Transactions on Knowledge and Data Engineering 34.5 (2020): 2401-2415.

Why Traditional Methods Fail

Distance Bias

Nodes that are distant in the network are automatically separated in the latent space, completely independent of their local, internal structural similarities.

ID-Based Walks

Popular embedding methods rely on traditional random walks with node IDs. This confines learning to specific neighborhoods rather than generalized structural patterns.

Disconnected Components

Traditional walks cannot traverse disconnected components. Consequently, nodes in entirely different graphs can never be assigned similar structural embeddings.

References:

- Ribeiro, Leonardo FR, Pedro HP Saverese, and Daniel R. Figueiredo. "struc2vec: Learning node representations from structural identity." Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining. 2017.
- Ahmed, Nesreen K., et al. "Role-based graph embeddings." IEEE Transactions on Knowledge and Data Engineering 34.5 (2020): 2401-2415.

Learn Role-based Embeddings

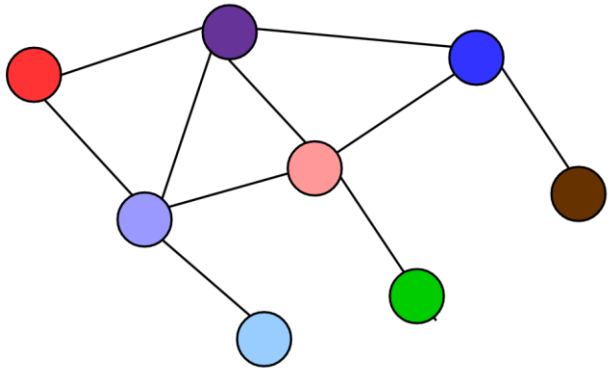
Goal: Find *d-dimensional* embeddings of nodes that preserve structural similarity.

Based on structural properties of nodes + *attributes (if any)*

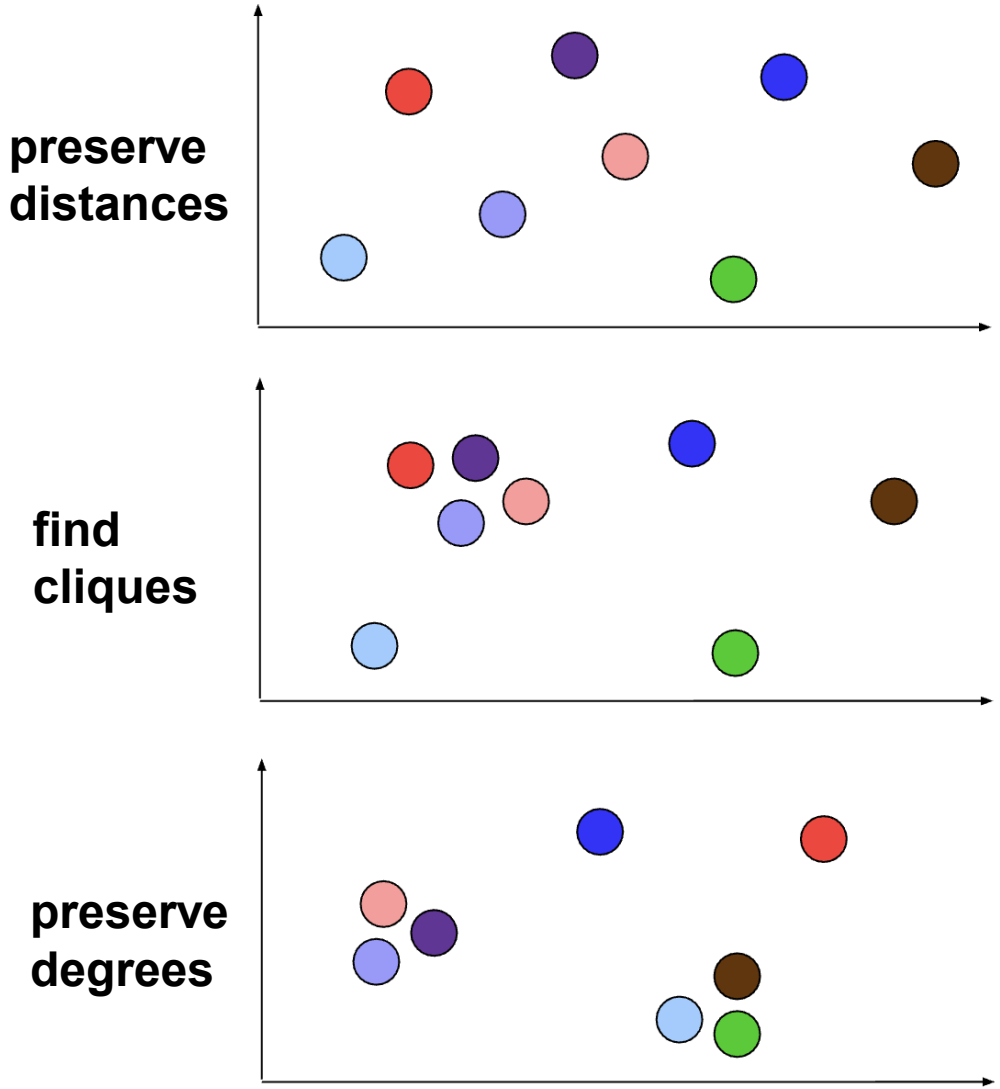
Reference: Ahmed, Nesreen K., et al. "Role-based graph embeddings." *IEEE Transactions on Knowledge and Data Engineering* 34.5 (2020): 2401-2415.

Node Representations

- Map network nodes into Euclidean space
  - aka. network embedding



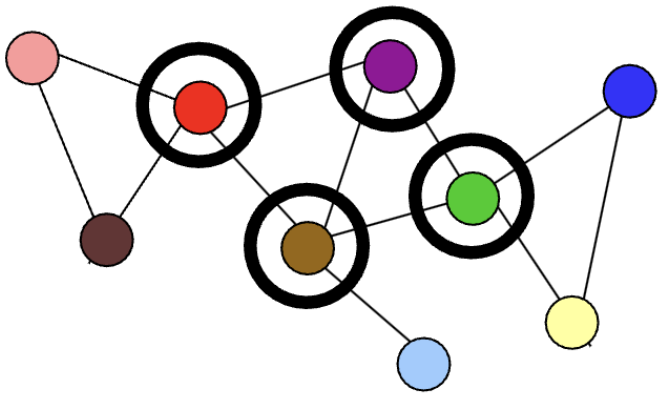
- Many ways to embednodes
- Right way depends onapplication



Leonardo F. R. Ribeiro et al: struc2vec: Learning Node Representations from Structural Identity

Structural Identity

- Nodes in networks have specific roles
  - E.g., individuals, web pages, proteins, etc
- Structural identity: identification of nodes based on network structure (no other attribute)
  - often related to role played by node
- **Automorphism**: strong structural equivalence



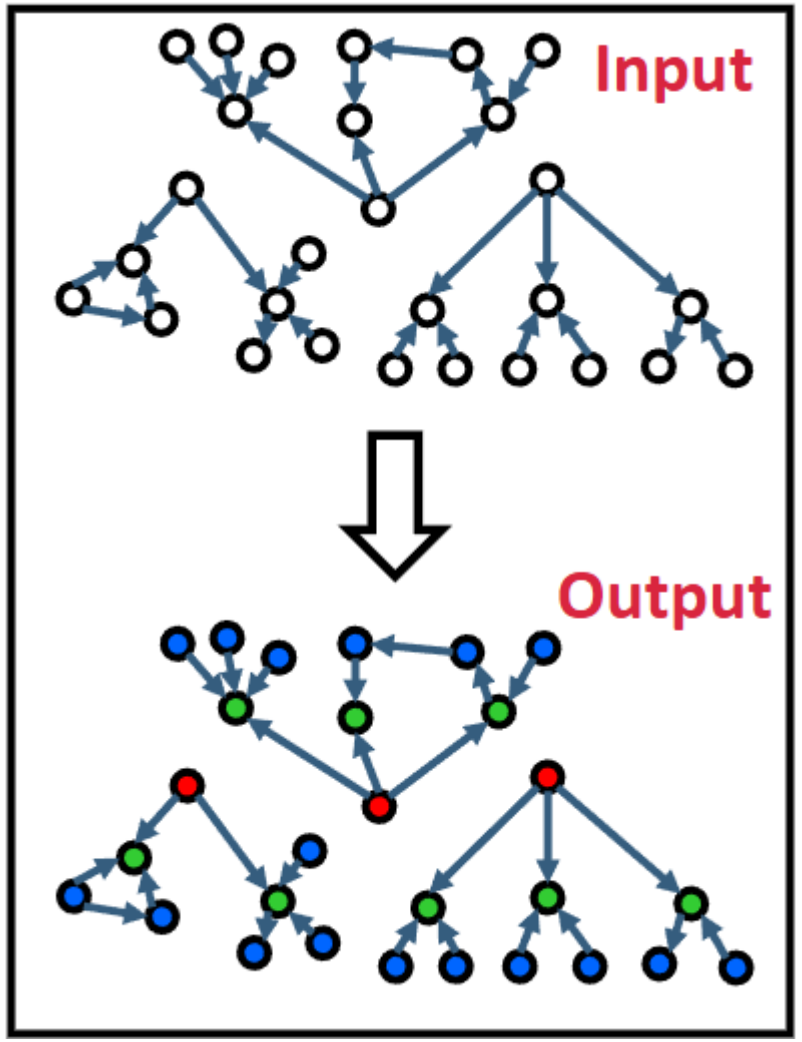
Red, Green: Automorphism
Purple, Brown: Structurally similar

Leonardo F. R. Ribeiro et al: struc2vec: Learning Node Representations from Structural Identity

RolX: Structural Role Extraction

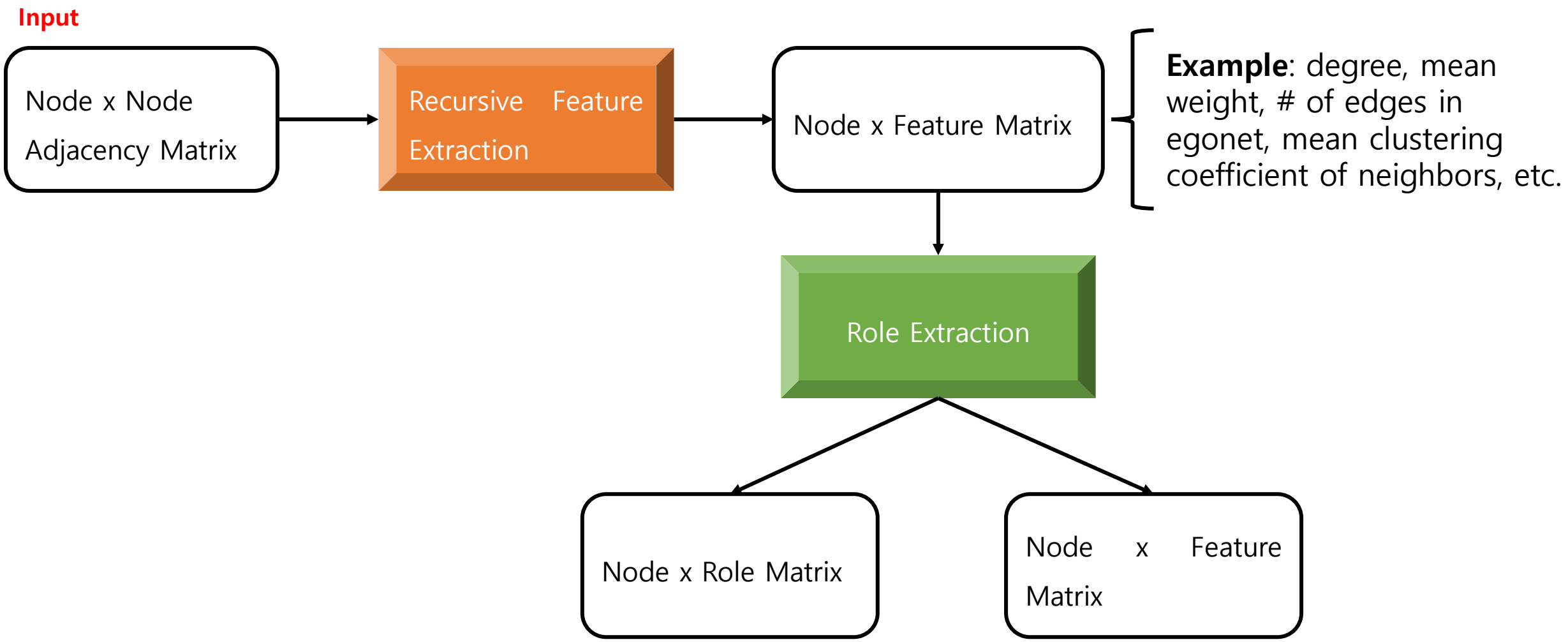
- RolX: Automatic discovery of nodes' structural roles in networks
  - Unsupervised learning approach
  - No prior knowledge required
  - Assigns a mixed-membership of roles to each node
  - Scales linearly in #(edges)

→ One of the earliest role-discovery methods.



Reference: Henderson, Keith, et al. "Rolx: structural role extraction & mining in large graphs." Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining. 2012.

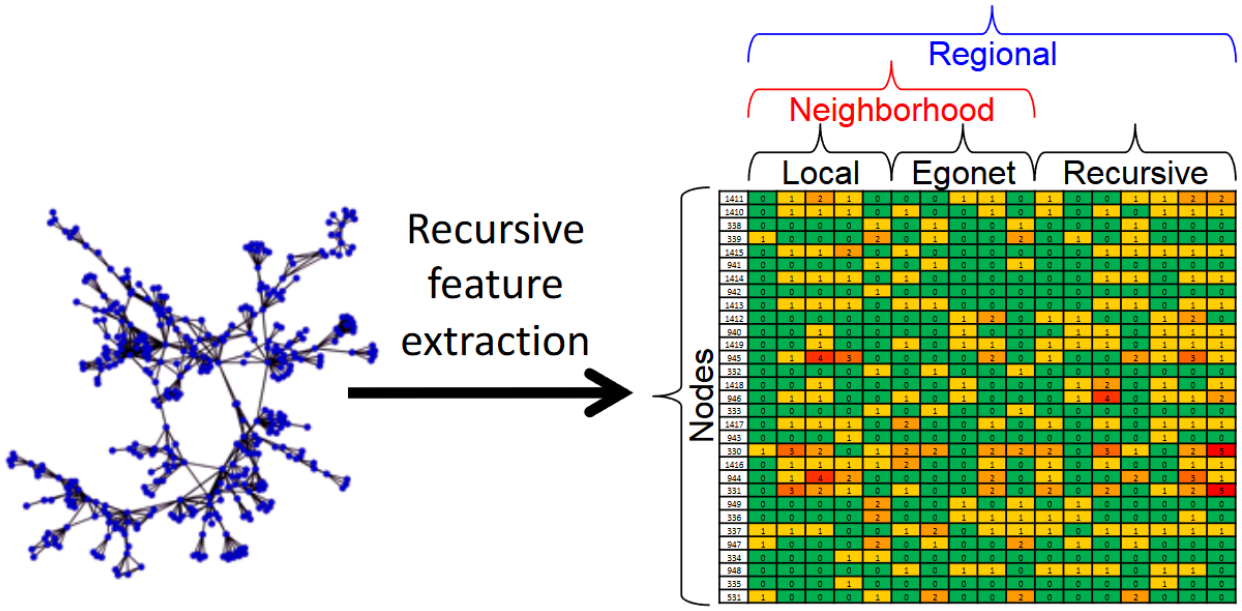
RolX: Structural Role Extraction

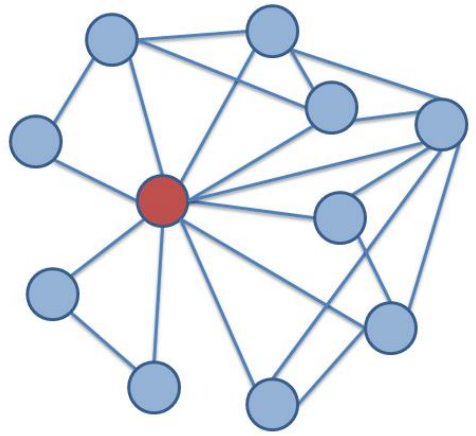


References:
[1] Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, cs224w.stanford.edu
[2] Henderson, Keith, et al. "Rolx: structural role extraction & mining in large graphs." Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining. 2012.

RolX: Structural Role Extraction

➢ **Recursive feature extraction** turns network connectivity into structural features





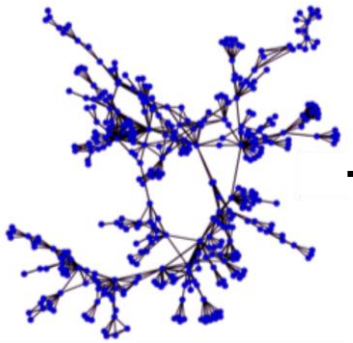
Egonet for red node

- **Neighborhood features**: What is a node's connectivity pattern?
- **Recursive features**: To what kinds of nodes is a node connected?

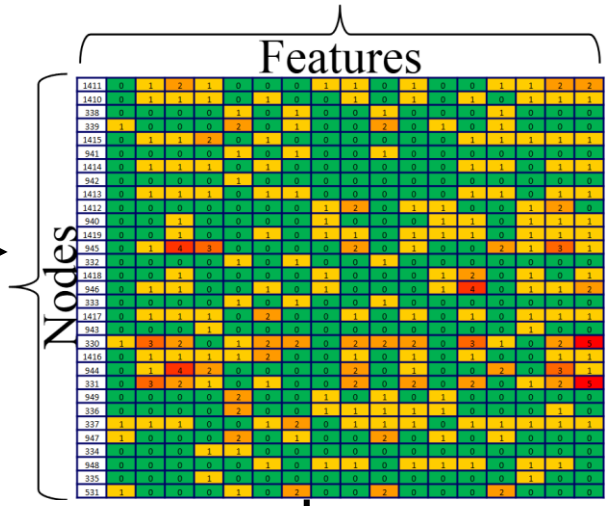
References:
[1] Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, cs224w.stanford.edu
[2] Henderson, Keith, et al. "Rolx: structural role extraction & mining in large graphs." Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining. 2012.

➢ **Recursive feature extraction** turns network connectivity into structural features

Input

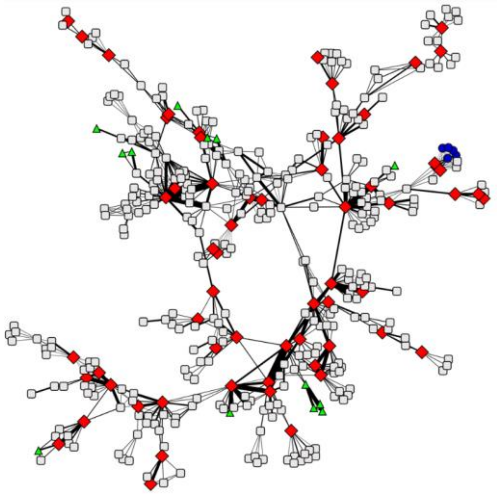


Recursive Feature Extraction



Cluster nodes based on extracted features

Output



RolX uses non-negative matrix factorization for clustering, MDL for model selection, and KL divergence to measure likelihood

References:
[1] Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, cs224w.stanford.edu
[2] Henderson, Keith, et al. "Rolx: structural role extraction & mining in large graphs." Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining. 2012.

Application: Structural Similarity

- **Task:** Cluster nodes based on their structural similarity
- **Two networks:**
  - Network science co-authorship network:
    - Nodes: Network scientists;
    - Edges: The number of co-authored papers
  - Political books co-purchasing network:
    - Nodes: Political books on Amazon;
    - Edges: Frequent co-purchasing of books by the same buyers
- Setup: For each network:
  - Use RolX to assign each node a distribution over the set of discovered, structural roles
  - Determine similarity between nodes by comparing their role distributions

Reference: Henderson, Keith, et al. "Rolx: structural role extraction & mining in large graphs." Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining. 2012.

Problems with RolX

But, RolX has the following limitations

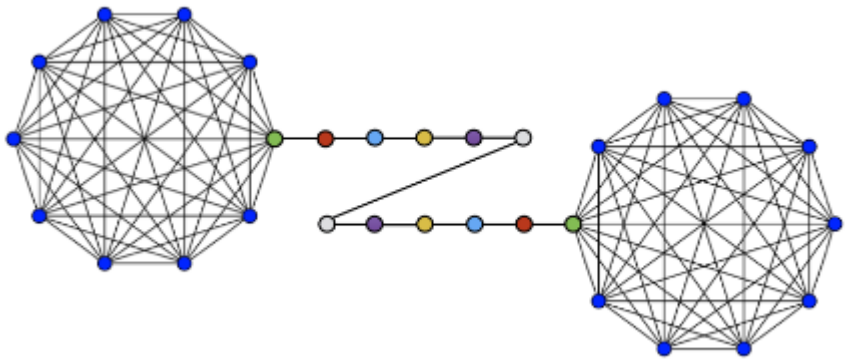
- **Missed Structural Equivalences:** By relying on basis vector assignments without explicitly considering node similarity or context, RolX is likely to miss node pairs that are structurally equivalent.

Reference: Ribeiro, Leonardo FR, Pedro HP Saverese, and Daniel R. Figueiredo. "struc2vec: Learning node representations from structural identity." Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining. 2017.

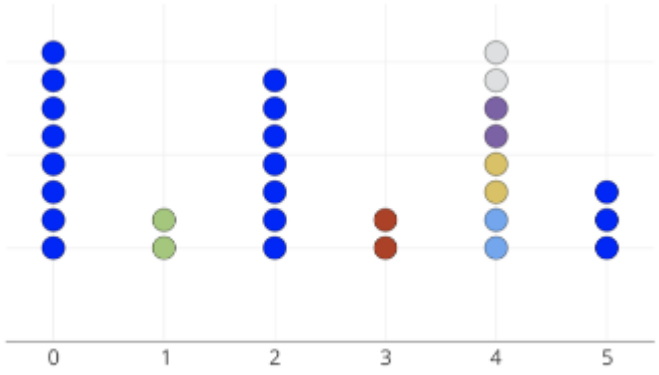
Problems with RolX

But, RolX has the following limitations

- **Missed Structural Equivalences**
- **Inconsistent Role Assignment**: In experiments with the barbell graph, RolX placed structurally identical nodes (from the same cliques) into three different roles, while grouping unrelated path nodes into a single role.



(a) Barbell Graph B(10, 10)



(b) *RolX*

Reference: Ribeiro, Leonardo FR, Pedro HP Saverese, and Daniel R. Figueiredo. "struc2vec: Learning node representations from structural identity." Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining. 2017.

Problems with RolX

But, RolX has the following limitations

- **Missed Structural Equivalences**
- **Inconsistent Role Assignment**
- **Failure in Symmetry Detection**: In the *mirrored* Karate network, RolX failed to consistently identify symmetry, placing many corresponding mirrored pairs into different roles and assigning leaders to separate categories.

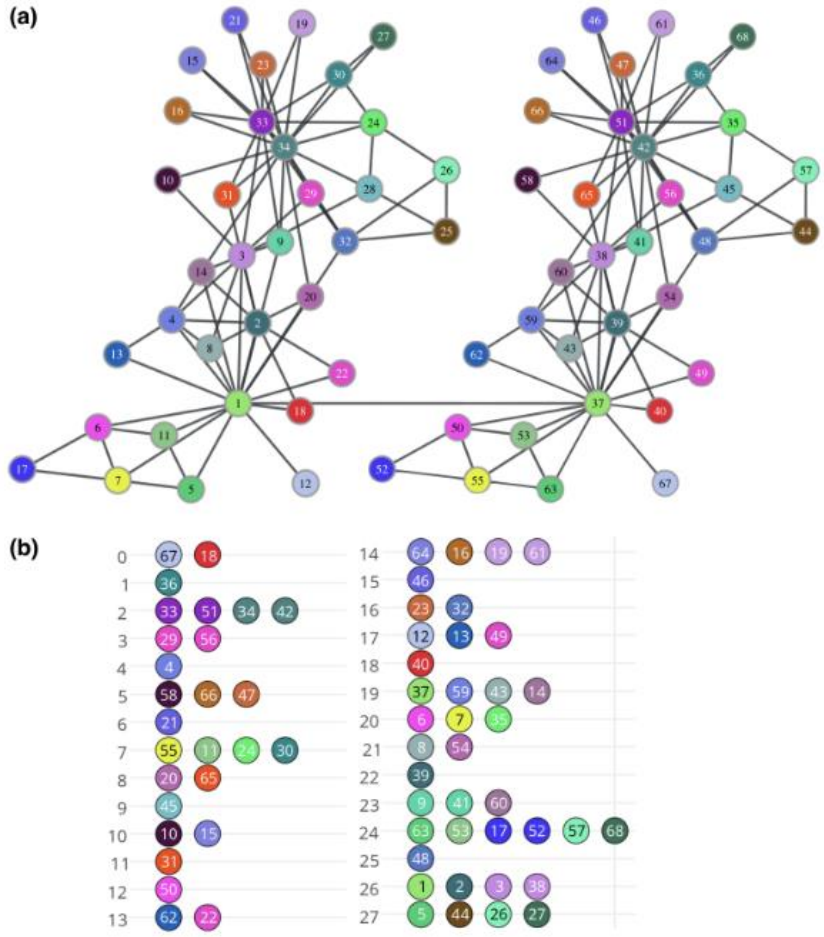


Figure: (a) Mirrored Karate network. Identical colors correspond to mirrored nodes. (b) Roles identified by RolX

Reference: Ribeiro, Leonardo FR, Pedro HP Saverese, and Daniel R. Figueiredo. "struc2vec: Learning node representations from structural identity." Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining. 2017.

struc2vec: Learning Node Representations from Structural Identity

- Idea: Based on structural identity
- Structural similarity does not depend on hop distance
  - Neighbour nodes can be different, far away nodes can be similar
- Structural identity as a hierarchical concept
  - Depth of similarity varies
- Flexible four step procedure
  - Operational aspect of steps are flexible

Leonardo F. R. Ribeiro et al: struc2vec: Learning Node Representations from Structural Identity

struc2vec: Step 1 - Structural Similarity

- $g(D_1,D_2)$: distance between two ordered sequences
  - Cost of pairwise alignment: max(a, b) / min(a, b) - 1
  - Optimal alignment by Dynamic Time Warping (DTW) in this framework

$s(R_0(u)) = 4$	$s(R_1(u)) = 1,3,4,4$	$s(R_2(u)) = 2,2,2,2$
$s(R_0(v)) = 3$	$s(R_1(v)) = 4,4,4$	$s(R_2(v)) = 1,2,2,2,2$
$g(.\ ,\ .) = 0.33$	$g(.\ ,\ .) = 3.33$	$g(.\ ,\ .) = 1$

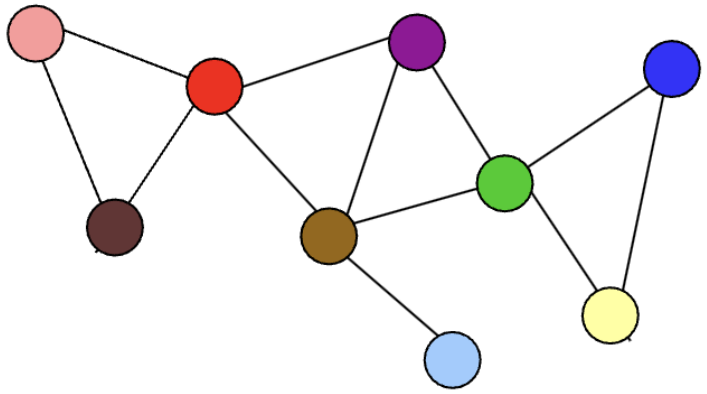
- $f_k(u,v)$: Structural distance between nodes u and v considering first k rings
  - $f_k(u,v) = f_{k-1}(u,v) + g(s(R_k(u)), s(R_k(v)))$

$f_0(u,v) = 0.33$ $f_1(u,v) = 3.66$ $f_2(u,v) = 4.66$

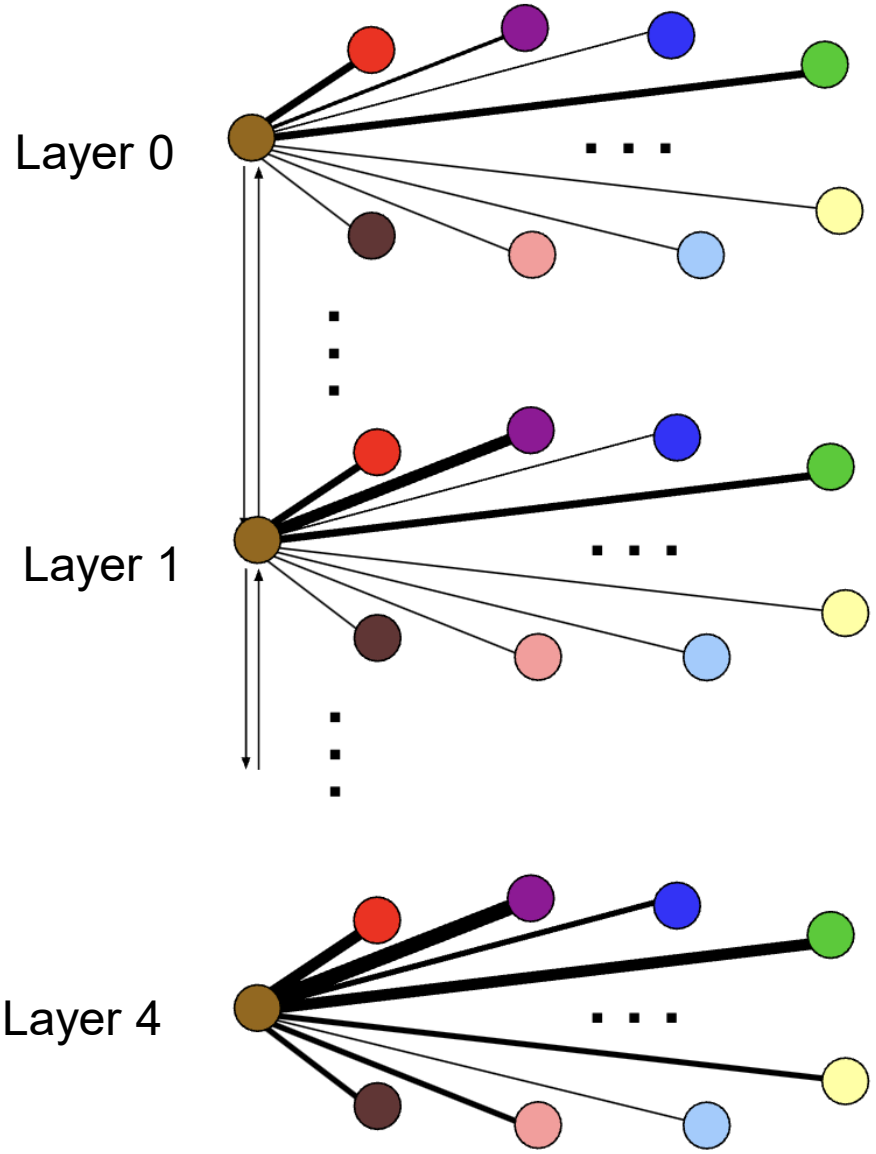
Leonardo F. R. Ribeiro et al: struc2vec: Learning Node Representations from Structural Identity

struc2vec: Step 2 - Multi-layer Graph

- Encodes structural similarity between all node pairs



- Each layer is weighted complete graph
  - Correspond to similarity hierarchies
- Edge weights in layer k
  - $w_k(u, v) = \exp\{-f_k(u, v)\}$
- Connect corresponding nodes in adjacent layers



Leonardo F. R. Ribeiro et al: struc2vec: Learning Node Representations from Structural Identity

struc2vec: Step 3 - Generate Context

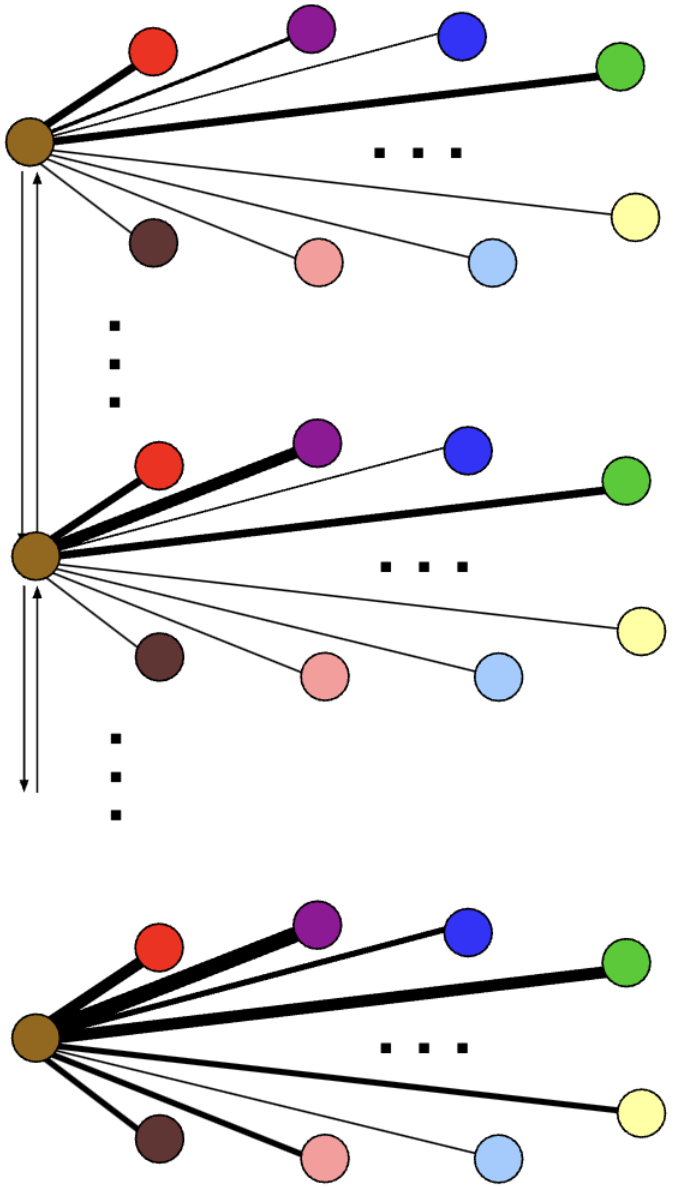
- Context generated by biased random walk (same as Node2vec)
  - Walk on multi-layer graph
- Walk in current layer with probability p
  - Choose neighbour according to edge weight
  - RW prefers more similar nodes
- Change layer with probability 1-p
  - Choose up/down according to edge weight
  - RW prefer layer with less similar neighbours

Leonardo F. R. Ribeiro et al: struc2vec: Learning Node Representations from Structural Identity

struc2vec: Step 4 – Learn Representation

- For each node, generate set of independent and relative short random walks
  - Context for node; sentences of a language

- Train a neural network to learn latent representation for nodes
  - Maximize probability of nodes within context
  - Skip-gram (Hierarchical Softmax) adopted



Leonardo F. R. Ribeiro et al: struc2vec: Learning Node Representations from Structural Identity

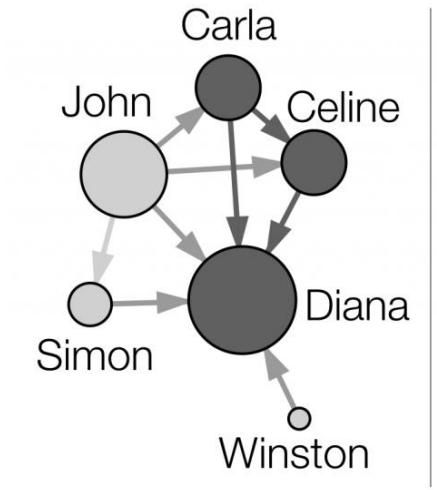
struc2vec: Optimization

- Reduce time to generate/store multi-layer graph and context for nodes:
  - Option 1: Reduce length of degree sequences
    - Use pairs (degree, number of occurrences)
  - Option 2: Reduce number of edges in multi-layer graph
    - Only log n neighbours per node
  - Option 3: Reduce number of layers in multi-layer graph
    - Fixed (small) number of layers
  - Scales quasi-linearly
    - Over 1 million nodes

Leonardo F. R. Ribeiro et al: struc2vec: Learning Node Representations from Structural Identity

role2vec: Learning Role-based Graph Embeddings

- Ideas: struc2vec and conventional methods (DeepWalk, Node2vec) learn in node representation
  - Role2Vec learns embeddings for "node types/roles" determined by node attributes/features
- Solution: "attributed random walks"
  - Map nodes to node types/roles using node attributes like motif counts, degrees, etc
  - Generate attributed random walks over the node types instead of node IDs
  - Finally, it learns embeddings for the node types rather than individual nodes

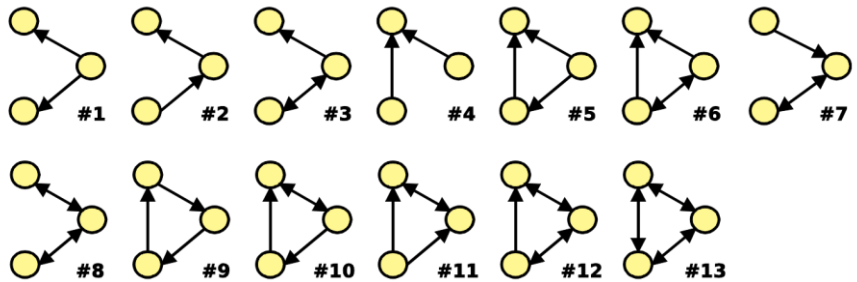


Nodes

Id,Label,Attribute
1,John,1
2,Carla,2
3,Simon,1
4,Celine,2
5,Winston,1
6,Diana,2

role2vec: Motifs

- Given a network, most of the time, some subgraphs are “overrepresented”
- A connected graph that has many occurrences in a network is called a motif of the network
- Assume set of occurrences G' in G is $occ_G(H)$
  - Cardinality of $occ_G(H)$ in G is frequent
  - How to know if G' is frequent in G?



⇒ Compute the probability that $occ_N(G') \geq occ_G(G')$ for a random network N

G' is said to be frequent in G if this probability is small enough

To compute this probability, we need to have a distribution over networks

role2vec: Algorithms

- **Input**: Graph G, node attribute matrix X, embedding dimension D, number of walks per node R, walk length L, context window size ω
- If X is not available, extract structural features like motif counts from the graph structure itself and use those as node attributes in X.
- Apply logarithmic binning to the node attribute values in X. Map nodes to node types/roles using a function $\phi(x)$ that takes the node attribute vector x as input
  - Two types of ϕ functions:
    - Simple functions like concatenation of attribute values
    - Low-rank matrix factorization of X
- Precompute random walk transition probabilities π. Generate R attributed random walks of length L for each node, using the node type mapping ϕ instead of node IDs
- Learn embeddings using stochastic gradient descent on the attributed random walks, optimizing the probability of observing the context node types in the walks
- **Output**: Learned embeddings for each node type w $\in$ W, where W is the set of node types found by ϕ

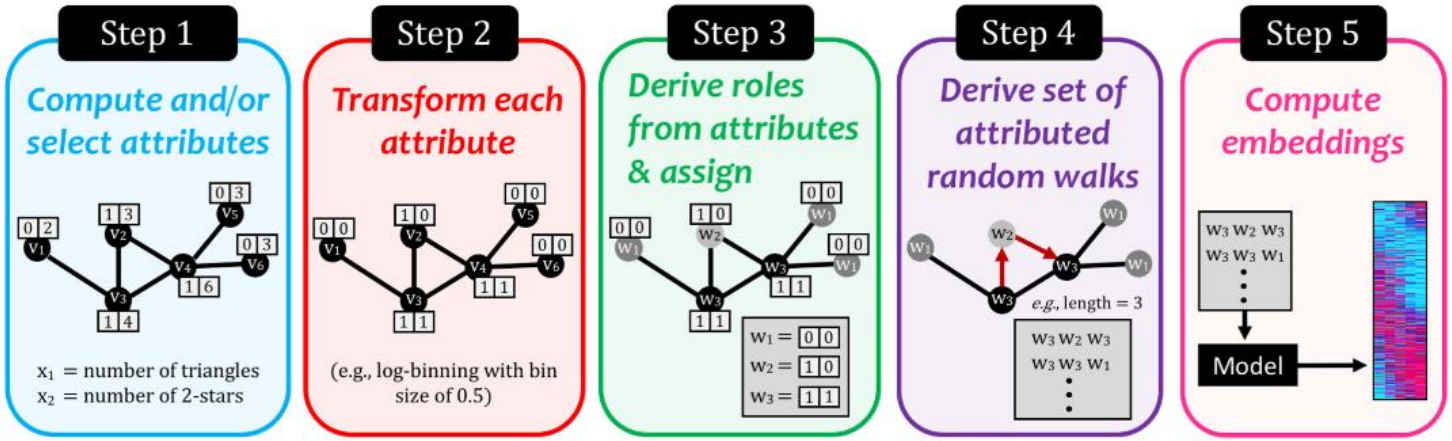
Nesreen K. Ahmed et al: Learning Role-based Graph Embeddings

role2vec: Algorithms

Algorithm 1. Role2Vec

procedure ROLE2VEC($G = (V, \mathrm{E})$ and X, embedding dimensions D, walks per node R, walk length L, context/window size ω)
  Initialize set of *attributed walks* $\mathcal{S}$ to $\emptyset$
  Extract (motif) features if needed and append to X
  Transform each attribute in X (e.g., using log binning)
  Map vertices to roles function $\Phi : \mathrm{x} \rightarrow w$
  Precompute transition probabilities π
  $G' = (V, E, \pi)$
  parallel for $j = 1, 2, \ldots, R$ **do** ▷ walks per node
    Set Π to be a random permutation of the nodes V
    for each $v \in \Pi$ in order **do**
      $S = \textsc{AttributedWalk}(G', \mathrm{X}, v, \Phi, L)$
      Add the *attributed walk* S to $\mathcal{S}$
  end parallel
  $\boldsymbol{\alpha} = \textsc{StochasticGradDescent}(\omega, D, \mathcal{S})$ ▷ **parallel**
  return the learned *role* embeddings $\boldsymbol{\alpha}$

procedure ATTRIBUTEDWALK(G', X, start node s, function Φ, L)
  Initialize attributed walk S to $[\Phi(\mathrm{x}_s)]$
  Set $i = s$ ▷ current node
  for $\ell = 1$ **to** $L - 1$ **do**
    $\Gamma_i =$ Set of the neighbors for node i
    $j = \textsc{AliasSample}(\Gamma_i, \pi)$ ▷ select node $j \in \Gamma_i$
    Append $\Phi(\mathrm{x}_j)$ to S
    Set i to be the current node j
  return attributed walk S of length L rooted at node s



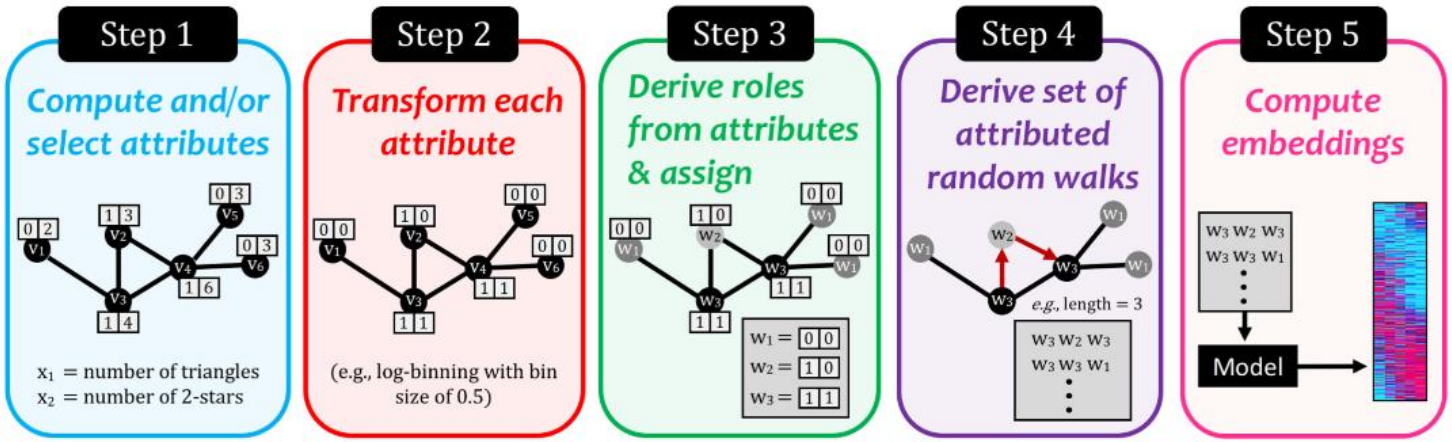
Nesreen K. Ahmed et al: Learning Role-based Graph Embeddings

role2vec: Algorithms

Algorithm 1. Role2Vec

procedure ROLE2VEC($G = (V, \mathrm{E})$ and X, embedding dimensions D, walks per node R, walk length L, context/window size ω)
  Initialize set of *attributed walks* $\mathcal{S}$ to $\emptyset$
  Extract (motif) features if needed and append to X
  Transform each attribute in X (e.g., using log binning)
  Map vertices to roles function $\Phi : \mathrm{x} \rightarrow w$
  Precompute transition probabilities π
  $G' = (V, E, \pi)$
  parallel for $j = 1, 2, \ldots, R$ **do** ▷ walks per node
    Set Π to be a random permutation of the nodes V
    for each $v \in \Pi$ in order **do**
      $S = \mathrm{ATTRIBUTEDWALK}(G', \mathrm{X}, v, \Phi, L)$
      Add the *attributed walk* S to $\mathcal{S}$
  end parallel
  $\boldsymbol{\alpha} = \mathrm{STOCHASTICGRADDESCENT}(\omega, D, \mathcal{S})$ ▷ **parallel**
  return the learned *role* embeddings $\boldsymbol{\alpha}$

procedure ATTRIBUTEDWALK(G', X, start node s, function Φ, L)
  Initialize attributed walk S to $[\Phi(\mathrm{x}_s)]$
  Set $i = s$ ▷ current node
  for $\ell = 1$ **to** $L - 1$ **do**
    $\Gamma_i =$ Set of the neighbors for node i
    $j = \mathrm{ALIASSAMPLE}(\Gamma_i, \pi)$ ▷ select node $j \in \Gamma_i$
    Append $\Phi(\mathrm{x}_j)$ to S
    Set i to be the current node j
  return attributed walk S of length L rooted at node s



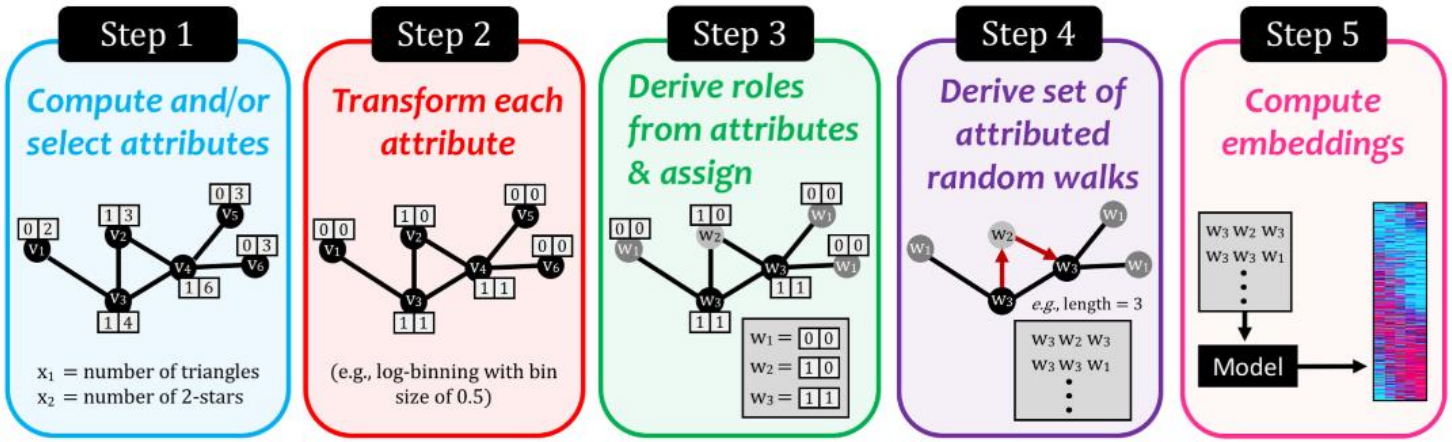
Nesreen K. Ahmed et al: Learning Role-based Graph Embeddings

role2vec: Algorithms

Algorithm 1. Role2Vec

procedure ROLE2VEC($G = (V, \mathrm{E})$ and X, embedding dimensions D, walks per node R, walk length L, context/window size ω)
  Initialize set of *attributed walks* $\mathcal{S}$ to $\emptyset$
  Extract (motif) features if needed and append to X
  Transform each attribute in X (e.g., using log binning)
  Map vertices to roles function $\Phi : \mathrm{X} \rightarrow w$
  Precompute transition probabilities π
  $G' = (V, E, \pi)$
  parallel for $j = 1, 2, \ldots, R$ **do** ▷ walks per node
    Set Π to be a random permutation of the nodes V
    for each $v \in \Pi$ in order **do**
      $S = \text{ATTRIBUTEDWALK}(G', \mathrm{X}, v, \Phi, L)$
      Add the *attributed walk* S to $\mathcal{S}$
  end parallel
  $\boldsymbol{\alpha} = \text{STOCHASTICGRADDESCENT}(\omega, D, \mathcal{S})$ ▷ **parallel**
  return the learned *role* embeddings $\boldsymbol{\alpha}$

procedure ATTRIBUTEDWALK(G', X, start node s, function Φ, L)
  Initialize attributed walk S to $[\Phi(\mathrm{x}_s)]$
  Set $i = s$ ▷ current node
  for $\ell = 1$ **to** $L - 1$ **do**
    $\Gamma_i =$ Set of the neighbors for node i
    $j = \text{ALIASSAMPLE}(\Gamma_i, \pi)$ ▷ select node $j \in \Gamma_i$
    Append $\Phi(\mathrm{x}_j)$ to S
    Set i to be the current node j
  return attributed walk S of length L rooted at node s



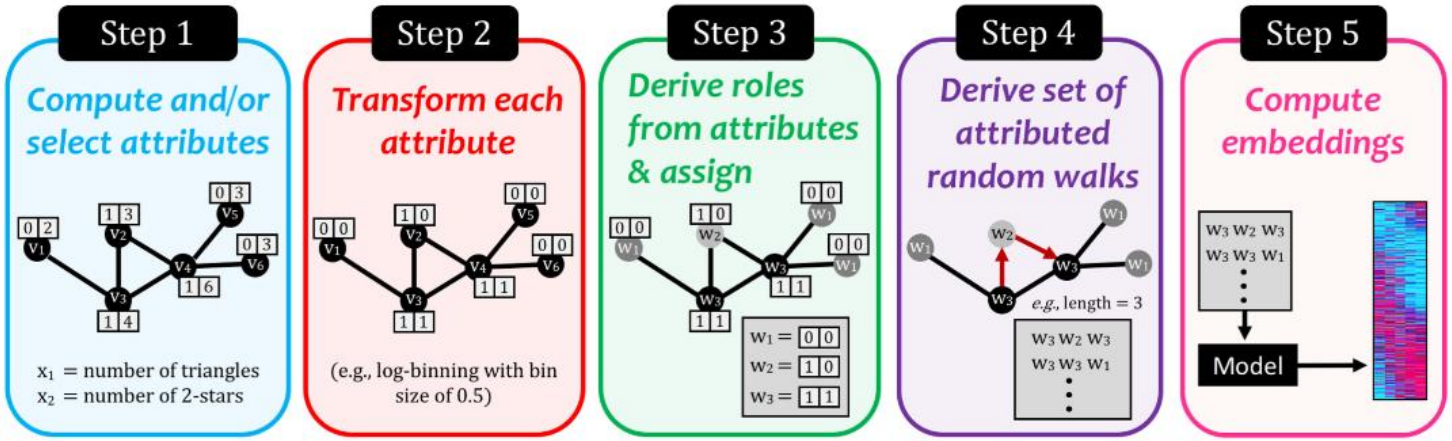
Nesreen K. Ahmed et al: Learning Role-based Graph Embeddings

role2vec: Algorithms

Algorithm 1. Role2Vec

procedure ROLE2VEC($G = (V, \mathrm{E})$ and X, embedding dimensions D, walks per node R, walk length L, context/window size ω)
  Initialize set of *attributed walks* $\mathcal{S}$ to $\emptyset$
  Extract (motif) features if needed and append to X
  Transform each attribute in X (e.g., using log binning)
  Map vertices to roles function $\Phi : \mathrm{x} \rightarrow w$
  Precompute transition probabilities π
  $G' = (V, E, \pi)$
  parallel for $j = 1, 2, \ldots, R$ **do** ▷ walks per node
    Set Π to be a random permutation of the nodes V
    for each $v \in \Pi$ in order **do**
      $S = \text{ATTRIBUTEDWALK}(G', \mathrm{X}, v, \Phi, L)$
      Add the *attributed walk* S to $\mathcal{S}$
  end parallel
  $\boldsymbol{\alpha} = \text{STOCHASTICGRADDESCENT}(\omega, D, \mathcal{S})$ ▷ **parallel**
  return the learned *role* embeddings $\boldsymbol{\alpha}$

procedure ATTRIBUTEDWALK(G', X, start node s, function Φ, L)
  Initialize attributed walk S to $[\Phi(\mathrm{x}_s)]$
  Set $i = s$ ▷ current node
  for $\ell = 1$ **to** $L - 1$ **do**
    $\Gamma_i =$ Set of the neighbors for node i
    $j = \text{ALIASSAMPLE}(\Gamma_i, \pi)$ ▷ select node $j \in \Gamma_i$
    Append $\Phi(\mathrm{x}_j)$ to S
    Set i to be the current node j
  return attributed walk S of length L rooted at node s



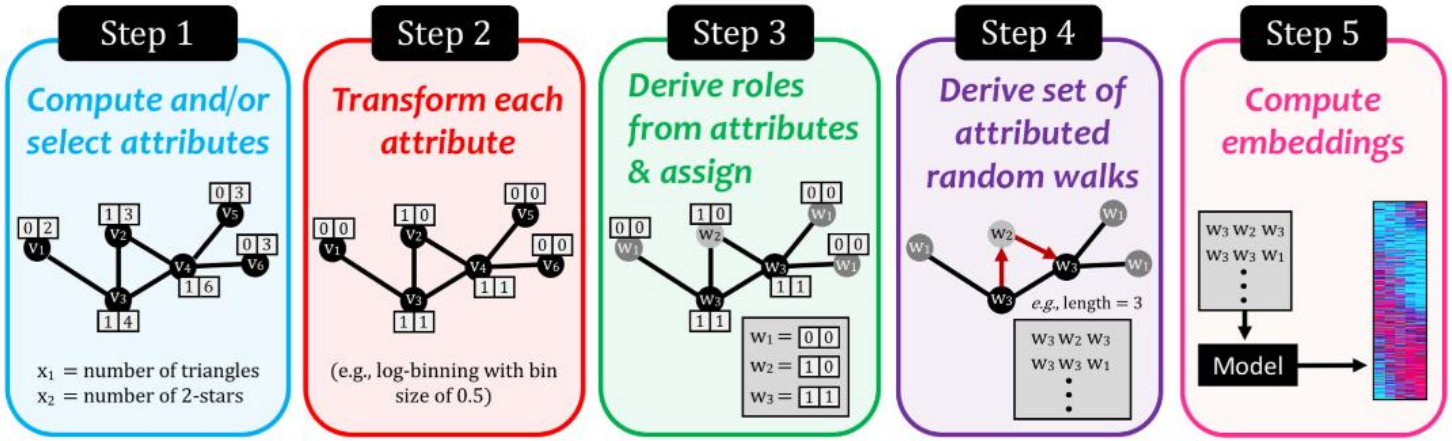
Nesreen K. Ahmed et al: Learning Role-based Graph Embeddings

role2vec: Algorithms

Algorithm 1. Role2Vec

procedure ROLE2VEC($G = (V, \mathrm{E})$ and X, embedding dimensions D, walks per node R, walk length L, context/window size ω)
  Initialize set of *attributed walks* $\mathcal{S}$ to $\emptyset$
  Extract (motif) features if needed and append to X
  Transform each attribute in X (e.g., using log binning)
  Map vertices to roles function $\Phi : \mathrm{x} \rightarrow w$
  Precompute transition probabilities π
  $G' = (V, E, \pi)$
  parallel for $j = 1, 2, \ldots, R$ **do** ▷ walks per node
    Set Π to be a random permutation of the nodes V
    for each $v \in \Pi$ in order **do**
      $S = \textsc{AttributedWalk}(G', \mathrm{X}, v, \Phi, L)$
      Add the *attributed walk* S to $\mathcal{S}$
  end parallel
  $\boldsymbol{\alpha} = \textsc{StochasticGradDescent}(\omega, D, \mathcal{S})$ ▷ **parallel**
  return the learned *role* embeddings $\boldsymbol{\alpha}$

procedure ATTRIBUTEDWALK(G', X, start node s, function Φ, L)
  Initialize attributed walk S to $[\Phi(\mathrm{x}_s)]$
  Set $i = s$ ▷ current node
  for $\ell = 1$ **to** $L - 1$ **do**
    $\Gamma_i =$ Set of the neighbors for node i
    $j = \textsc{AliasSample}(\Gamma_i, \pi)$ ▷ select node $j \in \Gamma_i$
    Append $\Phi(\mathrm{x}_j)$ to S
    Set i to be the current node j
  return attributed walk S of length L rooted at node s



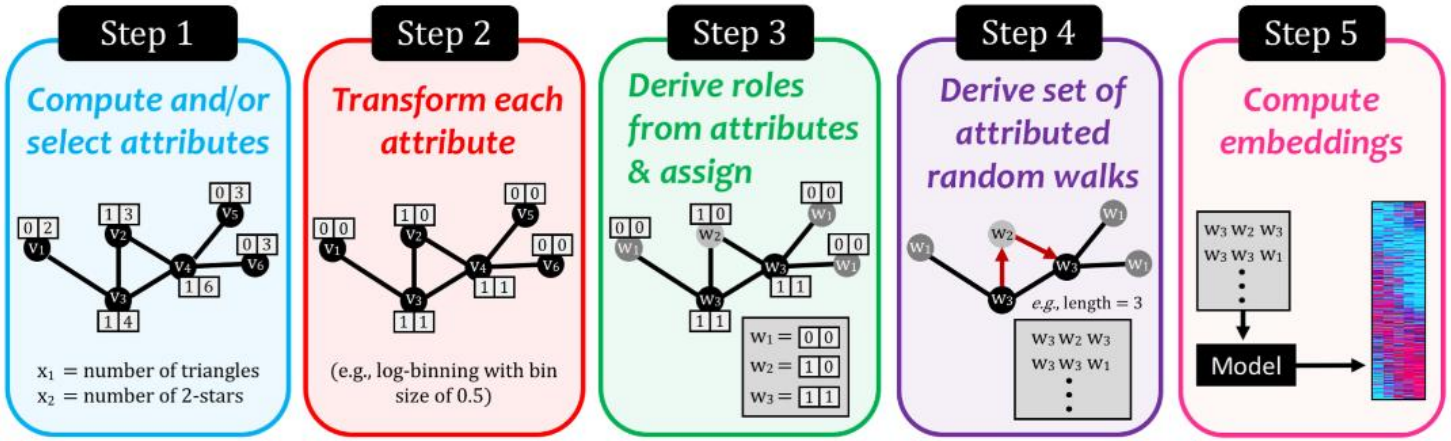
Nesreen K. Ahmed et al: Learning Role-based Graph Embeddings

role2vec: Algorithms

Algorithm 1. Role2Vec

procedure ROLE2VEC($G = (V, \mathrm{E})$ and X, embedding dimensions D, walks per node R, walk length L, context/window size ω)
  Initialize set of *attributed walks* $\mathcal{S}$ to $\emptyset$
  Extract (motif) features if needed and append to X
  Transform each attribute in X (e.g., using log binning)
  Map vertices to roles function $\Phi : \mathrm{x} \rightarrow w$
  Precompute transition probabilities π
  $G' = (V, E, \pi)$
  parallel for $j = 1, 2, \ldots, R$ **do** ▷ walks per node
    Set Π to be a random permutation of the nodes V
    for each $v \in \Pi$ in order **do**
      $S = \mathrm{ATTRIBUTEDWALK}(G', \mathrm{X}, v, \Phi, L)$
      Add the *attributed walk* S to $\mathcal{S}$
  end parallel
  $\boldsymbol{\alpha} = \mathrm{STOCHASTICGRADDESCENT}(\omega, D, \mathcal{S})$ ▷ **parallel**
  return the learned *role* embeddings $\boldsymbol{\alpha}$

procedure ATTRIBUTEDWALK(G', X, start node s, function Φ, L)
  Initialize attributed walk S to $[\Phi(\mathrm{x}_s)]$
  Set $i = s$ ▷ current node
  for $\ell = 1$ **to** $L - 1$ **do**
    $\Gamma_i =$ Set of the neighbors for node i
    $j = \mathrm{ALIASSAMPLE}(\Gamma_i, \pi)$ ▷ select node $j \in \Gamma_i$
    Append $\Phi(\mathrm{x}_j)$ to S
    Set i to be the current node j
  return attributed walk S of length L rooted at node s



Nesreen K. Ahmed et al: Learning Role-based Graph Embeddings

role2vec: Different from Previous Methods

- Mostly the same as Node2Vec, except their work are considered in terms of node attributes, not node representations

- There are many advantages of node attributed embedding:
  - Embeddings generalize to new nodes/graphs (inductive learning)
  - Better captures structural node roles
  - Space-efficient as it learns fewer embeddings for node types instead of all nodes
  - Supports attributed graphs

Nesreen K. Ahmed et al: Learning Role-based Graph Embeddings

Limitations of previous work:

❑ **RolX**

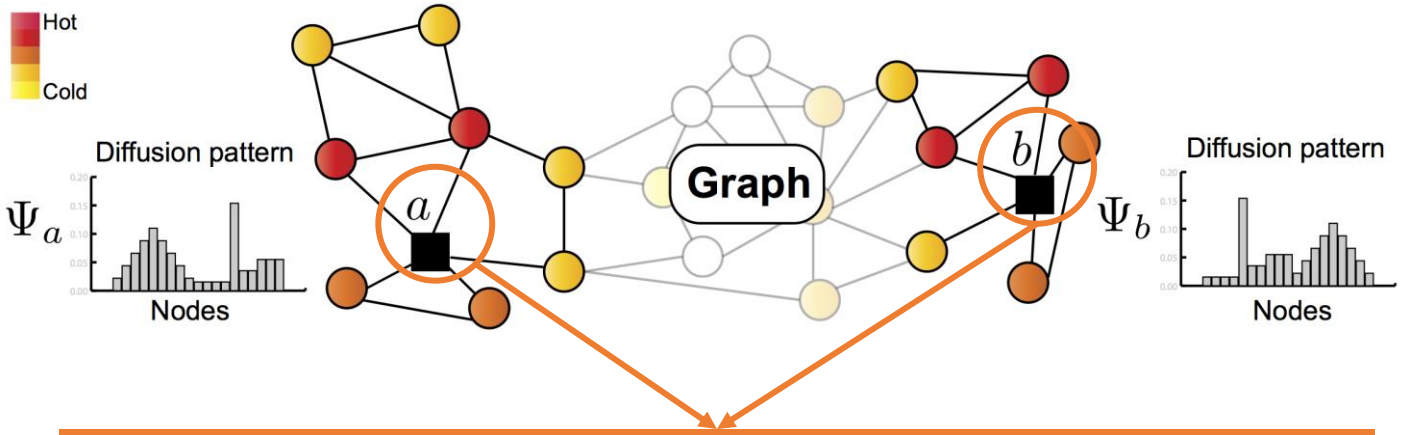
- **Manual Feature Dependency**: It relies on recursive extraction of manually selected features (like degree or number of triangles), which may require domain expertise.
- **Fixed Role Count**: It requires the user to pre-specify the number of distinct roles.

❑ **Struc2vec**

- **Computational Inefficiency**: It does not scale well to large graphs. In experiments, **struc2vec** took up to *260 seconds* on a network of *only 100 nodes*.
- **Heuristic Foundation**: It relies on heuristics for multilayered graph construction and random walk simulations
- **Poor Generalization**: It failed to generalize structural roles across different networks (e.g., airline networks), as its embeddings were often dominated by the specific graph identity rather than structural similarity.

Reference: Donnat, Claire, et al. "Learning structural node embeddings via diffusion wavelets." Proceedings of the 24 th ACM SIGKDD international conference on knowledge discovery & data mining. 2018.

GraphWave: Learning Structural Node Embeddings



Even if nodes a and b are far apart, their wavelet coefficient **distributions** will be nearly identical if their local neighborhoods are structurally similar.

- **Core Concept:** Nodes can perform the same structural "role" (e.g., a bridge or a hub) even if they are in distant, disconnected parts of a graph.
- **The GraphWave Approach**: Treats the ***diffusion pattern (heat wavelet)*** centered at each node as a probe of the local topology.

Reference: Donnat, Claire, et al. "Learning structural node embeddings via diffusion wavelets." Proceedings of the 24 th ACM SIGKDD international conference on knowledge discovery & data mining. 2018.

GraphWave: Learning Structural Node Embeddings

Algorithm 1 Learning structural embeddings in GRAPHWAVE.

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, scale s, evenly spaced sampling points $\{t_1, t_2, \ldots, t_d\}$
Output: Structural embedding $\chi_a \in \mathbb{R}^{2d}$ for every node $a \in \mathcal{V}$
Compute $\Psi = U g_s(\Lambda) U^T$ (Eq. (1))
for $t \in \{t_1, t_2, \ldots, t_d\}$ **do**
    Compute $\phi(t) =$ column-wise mean$(e^{it\Psi}) \in \mathbb{R}^N$
    for $a \in \mathcal{V}$ **do**
        Append Re$(\phi_a(t))$ and Im$(\phi_a(t))$ to χ_a

Spectral Wavelet Computation

GraphWave first computes the diffusion pattern for each node using a heat kernel. Given the graph Laplacian $L = D - A$, its eigenvector decomposition is $L = U\Lambda U^T$. The spectral graph wavelet Ψ_a for node a at scale s is defined as:

$$\Psi_a = U\mathrm{Diag}\big(g_s(\lambda_1), \ldots, g_s(\lambda_N)\big)U^T\delta_a$$

where $g_s(\lambda) = e^{-\lambda s}$ is the heat kernel and δ_a is a one-hot vector for node a. Each coefficient Ψ_{ma} represents the amount of energy node a propagates to node m.

Reference: Donnat, Claire, et al. "Learning structural node embeddings via diffusion wavelets." Proceedings of the 24 th ACM SIGKDD international conference on knowledge discovery & data mining. 2018.

GraphWave: Learning Structural Node Embeddings

Algorithm 1 Learning structural embeddings in GRAPHWAVE.

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, scale s, evenly spaced sampling points $\{t_1, t_2, \ldots, t_d\}$
Output: Structural embedding $\chi_a \in \mathbb{R}^{2d}$ for every node $a \in \mathcal{V}$
Compute $\Psi = U g_s(\Lambda) U^T$ (Eq. (1))
for $t \in \{t_1, t_2, \ldots, t_d\}$ **do**
    Compute $\phi(t) = \text{column-wise mean}(e^{it\Psi}) \in \mathbb{R}^N$
    for $a \in \mathcal{V}$ **do**
        Append $\text{Re}(\phi_a(t))$ and $\text{Im}(\phi_a(t))$ to χ_a

Note:

Instead of comparing wavelet vectors **directly** (which requires solving the node-alignment/isomorphism problem), GraphWave treats the coefficients $\{\Psi_{ma}\}_{m=1}^{N}$ as *samples from a probability distribution*. This allows the method to focus on *how* the diffusion spreads rather than *where* it spreads.

Reference: Donnat, Claire, et al. "Learning structural node embeddings via diffusion wavelets." Proceedings of the 24 th ACM SIGKDD international conference on knowledge discovery & data mining. 2018.

GraphWave: Learning Structural Node Embeddings

Algorithm 1 Learning structural embeddings in GRAPHWAVE.

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, scale s, evenly spaced sampling points $\{t_1, t_2, \ldots, t_d\}$
Output: Structural embedding $\chi_a \in \mathbb{R}^{2d}$ for every node $a \in \mathcal{V}$
Compute $\Psi = U g_s(\Lambda) U^T$ (Eq. (1))
for $t \in \{t_1, t_2, \ldots, t_d\}$ **do**
    Compute $\phi(t) =$ column-wise mean$(e^{it\Psi}) \in \mathbb{R}^N$
    for $a \in \mathcal{V}$ **do**
        Append Re$(\phi_a(t))$ and Im$(\phi_a(t))$ to χ_a

Empirical Characteristic Function

The distribution is embedded into a low-dimensional space using the **Empirical Characteristic Function**, which captures all moments of the distribution. For a node a, the Empirical Characteristic Function is calculated as:

$$\phi_a(t) = \frac{1}{N} \sum_{m=1}^{N} e^{it\Psi_{ma}}$$

This function is sampled at d evenly spaced points $\{t_1, \ldots, t_d\}$ to produce the final embedding

Reference: Donnat, Claire, et al. "Learning structural node embeddings via diffusion wavelets." Proceedings of the 24 th ACM SIGKDD international conference on knowledge discovery & data mining. 2018.

GraphWave: Learning Structural Node Embeddings

Algorithm 1 Learning structural embeddings in GRAPHWAVE.

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, scale s, evenly spaced sampling points $\{t_1, t_2, \ldots, t_d\}$
Output: Structural embedding $\chi_a \in \mathbb{R}^{2d}$ for every node $a \in \mathcal{V}$
Compute $\Psi = U g_s(\Lambda) U^T$ (Eq. (1))
for $t \in \{t_1, t_2, \ldots, t_d\}$ **do**
    Compute $\phi(t) =$ column-wise mean$(e^{it\Psi}) \in \mathbb{R}^N$
    for $a \in \mathcal{V}$ **do**
        Append $\mathrm{Re}(\phi_a(t))$ and $\mathrm{Im}(\phi_a(t))$ to χ_a

Final Embedding Construction

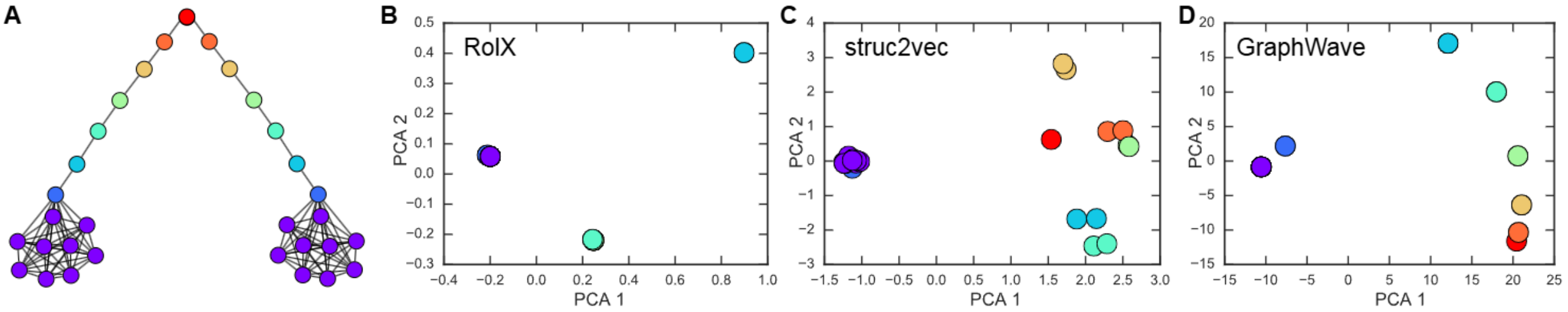
The **structural embedding χ_a** is formed by concatenating the real and imaginary parts of the sampled ECF:

$$\chi_a = \left[\mathrm{Re}\left(\phi_a(t_i)\right), \mathrm{Im}\left(\phi_a(t_i)\right)\right]_{t_1,\ldots,t_d}$$

For a multiscale representation, these vectors are concatenated across multiple scales s_j. The result is a $2dJ$-dimensional vector where J is the number of scales.

Reference: Donnat, Claire, et al. "Learning structural node embeddings via diffusion wavelets." Proceedings of the 24 th ACM SIGKDD international conference on knowledge discovery & data mining. 2018.

GraphWave: Learning Structural Node Embeddings



Structural Embeddings in a Barbell Graph

- GraphWave maps structurally equivalent nodes (same color) to identical points in the embedding space, resulting in overlapping points in the 2D PCA projection.
- Unlike struc2vec or RolX, GraphWave identifies the gradient-like pattern of roles along the chain connecting the two cliques.

Reference: Donnat, Claire, et al. "Learning structural node embeddings via diffusion wavelets." Proceedings of the 24 th ACM SIGKDD international conference on knowledge discovery & data mining. 2018.

네트워크 과학 연구실
NETWORK SCIENCE LAB

가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA